



AI-POWERED ROAD DAMAGE DETECTION & REPORTING SYSTEM

Comprehensive Technical & Conceptual Brief — AI Model Reference Document

Project Name	ROAD AI — AI
Lead Developer	Yanannathan Mueanhong (Cheemo), M.6, Varee Chiang Mai School
Co-Developer	Traisit Wuttichaikijtcharoen, M.6, Varee Chiang Mai School
Advisor	Sorawat Director: Waree
Competitions	NSC 2026 (28th National Software Contest, Category 23, code: 28P23N00470) • KMUTNB TIA 26th Innovation Award • CEDT Innovation Summit 2026
Current Status	Working prototype — laptop inference + Pi streaming established; ONNX offline deployment in progress
Document Date	June 2026

1. PROJECT OVERVIEW & PROBLEM STATEMENT

Thailand's road infrastructure suffers from widespread, under-documented damage. Local government agencies — **กรมทางหลวง** (Department of Highways), **เทศบาลนครเชียงใหม่** (Municipalities), and **อบต.** (Subdistrict Administrative Organisations) — rely on manual visual inspections that are slow, subjective, inconsistently documented, and expensive. Damage is frequently missed, incorrectly classified, or recorded on paper forms that are later lost or mis-filed.

ROAD AI automates the entire inspection pipeline: a camera-equipped Raspberry Pi 4 mounted on a survey vehicle captures road footage while driving; an AI model detects and classifies damage in real time; the system calculates the physical area of each defect, estimates repair cost in Thai Baht, pins the damage to a GPS map, and ultimately generates the official Thai government Form 1 repair-request document — all without human annotation.

Target Users & Deployment Context

- Thai Highway Department field engineers conducting road surveys
- Municipal maintenance crews (**เทศบาลนครเชียงใหม่** / **อบต.**) with limited technical resources
- Road inspection agencies requiring legally compliant Form 1 documentation
- Vehicles: pickup trucks, motorcycles, or any platform with a 12V power supply
- Environment: open roads, rural highways, urban streets in Chiang Mai and beyond
- Connectivity: unreliable 4G in rural areas — system must work fully OFFLINE

2. SYSTEM ARCHITECTURE

The system is split across two physical devices that communicate over a local Wi-Fi network (or direct hotspot). Each device has a distinct, well-defined role:

Layer	Device	Role
Capture	Raspberry Pi 4	USB webcam → OpenCV → JPEG frame → POST to laptop every 1 second
Inference	Laptop (current) Pi 4 (target)	Flask server receives frame, runs YOLO11 instance segmentation, returns detections
Storage	Laptop / Pi SD	Only frames WITH detections saved to disk. No-detection frames discarded immediately.
Presentation	Laptop browser	Single-page HTML5 dashboard: live feed, map, history, session management
Cloud (optional)	Remote server	Detections POSTed to cloud backend for long-term storage and cross-device access

2.1 Data Flow — Frame by Frame

Step 1 — Capture

PI's camera_worker thread reads frames from USB webcam at up to 30 fps using OpenCV VideoCapture. Resolution: 1280×720 for display, downscaled to 640×360 for inference.

Step 2 — Encode

Frame JPEG-encoded (quality 85) and POSTed via HTTP multipart/form-data to laptop server at POST /upload (port 5050) or processed locally by the inference_worker thread.

Step 3 — Infer

YOLO11 instance segmentation model runs on the frame. Returns bounding boxes (class + confidence + x/y/w/h) AND segmentation polygon (xy point list) for each detected damage.

Step 4 — Scale

Predictions scaled from inference resolution (640×360) back to display resolution (1280×720) using SX=2.0 and SY=2.0 scale factors.

Step 5 — Metrics

Physical dimensions calculated from pixel measurements using calibrated constants: PX_PER_CM_W = 27.06, PX_PER_CM_H = 29.58. Segmentation polygon area computed via OpenCV contourArea.

Step 6 — Filter

AUTO mode: detections below 60% confidence discarded. Same-class detections within 5-second cooldown window treated as the same damage instance (deduplication).

Step 7 — Event

Qualifying detections grouped into a Damage Event with: unique ID, class, severity, cost_thb, area_m2, GPS coordinates (real or simulated), annotated thumbnail (320×180 JPEG).

Step 8 — Store

Event appended to in-memory deque (maxlen=300 events, maxlen=500 flat detections). Annotated frame saved to disk ONLY if detections present — empty frames discarded.

Step 9 — Stream

output_frame (annotated) served as MJPEG stream at /video_feed. Frontend polls /detections, /events, /stats every second.

Step 10 — Map

Each event stamped with GPS lat/lng. On survey map, events rendered as colored pins (red=high, orange=medium, yellow=low severity) along the survey path polyline.

3. AI MODEL — YOLO11 INSTANCE SEGMENTATION

The core intelligence of ROAD AI is a YOLO11 instance segmentation model trained and hosted via Roboflow. Unlike simple object detection (bounding boxes only), instance segmentation produces a precise polygon outline of each damage area, enabling accurate physical area measurement in square centimetres and square metres.

Model architecture	YOLO11 Instance Segmentation (Ultralytics)
Training platform	Roboflow (cloud training + hosted inference API)
Inference endpoint	POST <code>http://localhost:9001/{workspace}/workflows/{workflow_id}</code>
Input format	Base64-encoded JPEG, 640×360 pixels
Output format	JSON: bounding box predictions + segmentation polygon point lists
Current accuracy	83.1% mAP@50
Classes trained	2 currently: Pothole, Longitudinal Crack
Classes planned	6 total: + Transverse Crack, Alligator Crack, Surface Damage, Edge Break
Confidence threshold	60% minimum (configurable via <code>.env MIN_CONFIDENCE</code>)
Dedup cooldown	5 seconds per class (configurable via <code>.env DEDUP_COOLDOWN</code>)
Inference resolution	640×360 (upscaled ×2 for 1280×720 display rendering)
Offline target	ONNX export for on-device Pi inference without internet

3.1 Roboflow Workflow (Current Cloud Pipeline)

The current implementation calls a Roboflow custom workflow that chains two model outputs: (1) a bounding box detection model ('predictions') and (2) an instance segmentation model ('model_1_predictions'). Both outputs are returned in a single API call. This architecture allows the system to simultaneously draw tight bounding boxes for labelling and segmentation masks for area measurement.

3.2 ONNX Offline Plan (Critical Next Step)

The single biggest known risk to field deployment is internet dependency. Rural Thai roads frequently have no reliable 4G signal. The planned solution is to export the trained YOLO11 model from Roboflow as an ONNX file and run inference directly on the Raspberry Pi 4 using the ONNX Runtime library. This eliminates the Roboflow API call entirely and reduces per-frame latency from ~200-400ms (cloud round-trip) to an estimated 80-200ms (on-device, no network).

- Export: Roboflow UI → Versions → Export → ONNX format → download `best.onnx`
- Runtime: `pip install onnxruntime` (CPU) or `onnxruntime-gpu` (if Pi has accelerator)
- Replace `infer_frame()` in `roadai.py` with local ONNX `session.run()` call
- Expected Pi 4 throughput: 3-8 FPS on CPU (sufficient for 1-frame/sec survey capture)
- Fallback: if Pi FPS insufficient, keep inference on laptop, Pi streams only

4. DAMAGE CLASSIFICATION, METRICS & COST ESTIMATION

Every detected damage instance is automatically classified by type, measured physically, assigned a severity level, and given a Thai Baht repair cost estimate. This transforms raw AI detections into actionable maintenance data.

4.1 Damage Classes & Repair Costs

Damage Class	THB / m ²	Notes
Pothole	850	Most expensive — requires asphalt excavation and fill
Longitudinal Crack	320	Surface sealing sufficient in most cases
Transverse Crack	320	Similar treatment to longitudinal; planned class
Alligator Crack	480	Structural fatigue; requires base repair
Surface Damage	560	Ravelling / rutting; overlay treatment
Default (unknown)	400	Fallback when class not in table

4.2 Physical Measurement Pipeline

Pixel-to-centimetre conversion uses two calibration constants derived from a known reference object filmed at standard camera height:

PX_PER_CM_W = 27.06 (pixels per centimetre, horizontal)

PX_PER_CM_H = 29.58 (pixels per centimetre, vertical)

Bounding box area (fallback): $\text{width_px} / \text{PX_PER_CM_W} \times \text{height_px} / \text{PX_PER_CM_H} = \text{area_cm2}$. Segmentation area (preferred): $\text{OpenCV contourArea() on the polygon point list} \rightarrow \text{area_px2} / (\text{PX_PER_CM_W} \times \text{PX_PER_CM_H}) = \text{area_cm2}$. Both converted to m2 by dividing by 10,000. Cost = $\text{area_m2} \times \text{cost_per_m2}$.

4.3 Severity Classification

Severity	Area Threshold	Map Pin Colour	Meaning
Low	< 100 cm ²	Yellow	Minor surface defect; monitor only
Medium	100–499 cm ²	Orange	Significant damage; schedule for repair
High	>= 500 cm ²	Red	Urgent; safety risk; immediate action needed

5. BACKEND — roadai.py

The backend is a multi-threaded Python Flask application (Python 3.12, Flask 3.x) that runs on the laptop at port 5000. It manages three concurrent threads and exposes a comprehensive REST API to the frontend.

5.1 Thread Architecture

camera_worker

Reads frames from USB webcam via OpenCV at 30 fps (1280×720). Applies current annotations overlay and writes to `output_frame` for streaming. Runs independently of inference pace.

inference_worker

In AUTO mode: continuously takes `latest_frame`, sends to Roboflow API (or local ONNX), receives predictions, filters them, builds damage events, updates shared state. Idles in MANUAL mode.

Flask main thread

Serves all HTTP endpoints. `generate_stream()` yields MJPEG frames from `output_frame` at 30 fps. All shared state access protected by a single `threading.Lock()`.

5.2 Detection Modes

AUTO mode — `inference_worker` runs continuously. Each qualifying detection (confidence $\geq 60\%$, not a duplicate within 5s) creates a Damage Event automatically. Designed for driving surveys where the operator focuses on the road.

MANUAL mode — `inference_worker` idles. The operator clicks 'CAPTURE & DETECT' in the UI, which calls `POST /capture`. The current frame is frozen, inference runs once, results returned immediately with an annotated preview image.

5.3 Session (Survey) Management

Surveys are wrapped in Sessions. An operator starts a session (`POST /session/start` with name + GPS start point), drives the route, then ends it (`POST /session/end`). The system tracks: GPS path (array of lat/lng points), all damage events with their positions, total distance (km), cumulative cost, duration. Multiple sessions are stored in memory and displayed as distinct coloured paths on the map.

5.4 API Endpoints Reference

Endpoint	Method	Purpose
<code>/video_feed</code>	GET	MJPEG live camera stream (30 fps)
<code>/detections</code>	GET	Latest frame's detection list with metrics
<code>/events</code>	GET	Damage event history (lightweight, no thumbnails by default)
<code>/events?include_thumb=1</code>	GET	Same as above but with base64 annotated thumbnails embedded
<code>/event/</code>	GET	Full event detail including all sub-detections + thumbnail
<code>/event//delete</code>	POST	Remove a false positive from history and all sessions
<code>/stats</code>	GET	Session summary: total detections, class counts, cost, area, uptime
<code>/history</code>	GET	Full flat detection history (all frames, not grouped into events)
<code>/mode</code>	GET	Get current detection mode (auto/manual)
<code>/mode</code>	POST	Set detection mode: <code>{mode: 'auto'}</code> or <code>{mode: 'manual'}</code>

/capture	POST	Manual capture: freeze frame, run inference, return annotated result
/clear	POST	Reset session detection history and counters
/session/start	POST	Begin new survey: {name, lat, lng}
/session/end	POST	End active survey, compute distance, finalise session
/session/current	GET	Status of the currently active survey
/sessions	GET	All surveys (newest first) as map-ready summaries
/session/	GET	Full detail for one survey
/session//type/	GET	All events of a specific damage type within a survey (with thumbs)
/session//delete	POST	Delete an entire survey from memory
/mock-locations	GET	8 preset Chiang Mai GPS points for testing without real GPS hardware

6. FRONTEND — index.html

The frontend is a single self-contained HTML5 file (no build step, no npm, no framework — runs directly in a browser). It has a dark cyberpunk aesthetic (background #0a0a0a, accent #c8f135 lime-green) and is built with vanilla JavaScript + Leaflet.js for mapping. It polls the Flask backend every second for live data.

Survey Map View

The main dashboard. A full-viewport Leaflet.js map shows all completed and active survey routes as coloured polylines (each session gets a unique colour: cyan, magenta, lime, orange...). Start and end points marked with  and  icons. Each damage event shown as a coloured circle marker sized by severity. Clicking a marker shows a popup with class, confidence, cost, and severity. A live sidebar shows current session stats. Operator can start/end surveys, pick GPS start location from a dropdown of preset Chiang Mai road coordinates.

Live Detection View

Two-panel layout. Left: MJPEG stream from /video_feed with animated corner brackets and a REC indicator. Right sidebar: real-time metrics from the latest detection — damage class, confidence percentage, bounding box dimensions (cm × cm), segmentation area (cm² and m²), estimated repair cost (THB), severity badge. Metrics update every second. Mode toggle (AUTO / MANUAL) in the header.

Manual Capture View

When in MANUAL mode, a large 'CAPTURE & DETECT' button triggers POST /capture. The UI freezes the annotated frame as a static preview image and displays the full detection breakdown. Designed for stationary inspections where the operator wants deliberate control over when inference fires.

Damage History View

A scrollable grid of all damage events from the current session. Each card shows: annotated thumbnail (320×180), damage class badge, severity indicator, confidence %, area (m²), and cost (). Cards filterable by severity or class. Operator can delete false positives (calls POST /event//delete). Newest events appear first.

Session Detail View

Activated by clicking a session on the map. Shows the survey's summary statistics: total events, total cost (THB), total area (m²), distance driven (km), duration. A breakdown table shows count by damage type. Each damage type row is clickable → opens Gallery View.

Damage Gallery View

Shows all photos of one damage type within one session. Full annotated thumbnails in a grid. Useful for preparing documentation or checking a specific damage category across an entire route.

6.1 Offline Demo Mode

If the frontend cannot reach the Flask backend (HTTP error or no connection), it automatically switches to Demo Mode. A warning pill appears in the header. All five views remain fully functional, populated with realistic fake data: multiple sessions, multiple damage types, GPS paths across Chiang Mai roads, cost breakdowns. This ensures the system can be demonstrated at competitions or meetings without requiring a live camera connection.

7. HARDWARE STACK

Edge compute	Raspberry Pi 4 Model B (4GB RAM recommended) — Ubuntu 22.04 LTS aarch64, Python 3.11
Camera	Logitech USB webcam (1080p capable, used at 1280×720 for balance of quality vs. bandwidth). OpenCV VideoCapture index 0 or 1.
Dev machine	Windows 11 laptop — Python 3.12, Flask 5050/5000, VS Code with Remote-SSH extension connected to Pi
GPS module	NEO-6M UART GPS (acquired, not yet wired) — will replace GPS simulation in advance_gps()
Connectivity	4G LTE USB dongle (status: not yet tested) — for cloud upload in field. System designed to work fully offline without it.
Power	Plan: 12V→5V USB-C stepdown from vehicle power or portable power bank (min 10,000 mAh for 4+ hour survey)
Mounting	Planned: dashcam-style mount on windscreen or external roof mount for clear downward road view
Network (dev)	Both Pi and laptop on same Wi-Fi network. Pi IP: 192.168.1.109. Laptop IP found via ipconfig/ifconfig.

7.1 Pi Streaming Pipeline (Current Implementation)

pi_capture.py runs on the Pi and sends frames to the laptop. It opens the webcam with OpenCV, captures one frame per second, JPEG-encodes it (quality 85), and POSTs it to `http://5050/upload` as multipart form data. receiver_server.py on the laptop accepts these frames. Connection errors are caught gracefully — the Pi logs the error and retries on the next cycle without crashing. If webcam index 0 fails, the script automatically tries index 1.

8. CURRENT STATE & KNOWN ISSUES

What is Working Right Now

- Raspberry Pi 4 set up with Ubuntu 22.04 LTS, SSH accessible from laptop
- USB webcam streaming from Pi to laptop via `pi_capture.py` → `receiver_server.py` over local Wi-Fi
- YOLO11 inference pipeline in `roadai.py` functional on laptop webcam (Roboflow cloud mode)
- 83.1% mAP@50 on 2 trained classes (Pothole, Longitudinal Crack)
- Full 5-view HTML dashboard working in browser with live data and demo mode
- Session management, GPS simulation, coloured route map, damage history, gallery view
- Cost estimation (THB) and severity classification for all detected events
- Annotated thumbnail generation and storage for each damage event
- False positive deletion, session deletion, auto/manual mode switching
- VS Code Remote-SSH workflow established for coding directly on Pi from laptop

Known Issues & Gaps

CRITICAL — Offline inference	System still calls Roboflow cloud API. Fails without internet. Fix: ONNX export + on-device inference. This is the #1 priority.
CRITICAL — No real GPS	GPS position is simulated with <code>advance_gps()</code> (random walk ~10m steps). NEO-6M module acquired but not wired. Positions on map are approximate/fake.
HIGH — No database	All detections and sessions stored in Python deque in RAM. Lost on restart. Fix: SQLite with SQLAlchemy for persistence across reboots.
HIGH — Form 1 not implemented	Thai government repair request form (Form 1 / ■■■■■■■■■■ 1) not yet generated. This is a key deliverable for the target agencies.
MEDIUM — Map tile dependency	Leaflet uses CARTO CDN tiles. Black map if CDN blocked. Fix: local tile server or offline tile cache.
MEDIUM — 4G not tested	4G dongle cloud upload path unverified in field conditions.
LOW — Pi FPS unknown	Inference speed on Pi hardware not benchmarked. May need FPS optimisation or relay model to laptop.
LOW — Single camera angle	One webcam captures one view. Wider coverage may require multiple cameras or panoramic lens.

9. DEVELOPMENT ROADMAP

1 ONNX Export & Offline Inference

Export YOLO11 model from Roboflow to ONNX. Integrate onnxruntime into roadai.py. Replace infer_frame() HTTP call with local session.run(). Benchmark FPS on Pi. Target: fully offline inference with no cloud dependency.

2 Real GPS Integration (NEO-6M)

Wire NEO-6M GPS module to Pi via UART (TX→Pi RX, RX→Pi TX, 3.3V, GND). Install gpsd + pyserial. Replace advance_gps() simulation with real NMEA sentence parsing. Add ~10m GPS jitter filtering (Kalman or moving average).

3 SQLite Database Persistence

Replace in-memory dequeues with SQLAlchemy + SQLite. Schema: sessions table, events table, detections table. Surveys survive reboots. Add export endpoints for JSON and CSV. Critical for real-world deployments.

4 Form 1 PDF Generation

Thai government repair request form (■■■■■■■■■■ 1). Pre-fill from session data: road name, date, GPS coordinates, damage type table, estimated costs, total area. Generate as PDF using reportlab. Operator reviews and submits digitally.

5 Expand Model Classes

Train YOLO11 on 4 additional damage types: Transverse Crack, Alligator Crack, Surface Damage (ravelling/rutting), Edge Break. Source training data from Thai road surveys. Target: 6-class model at $\geq 80\%$ mAP@50.

6 4G Cloud Upload & Sync

Test 4G dongle in field. Implement background sync queue: detections POST to cloud backend when connectivity available. Handle intermittent connection gracefully (queue, retry, deduplicate).

7 Field Trial — Chiang Mai Roads

Mount hardware in vehicle. Survey 3-5 real roads around Chiang Mai (Huay Kaew Rd, Nimman Rd, Suthep Rd target routes). Collect ground truth vs. AI detection comparison. Document accuracy, missed detections, false positives.

8 Competition Submissions

NSC 2026 submission (Category 23) — deadline tracked. KMUTNB TIA 26th preparation. CEDT Innovation Summit 2026 poster and demo materials. Prepare 3-minute pitch video demonstrating end-to-end pipeline from driving survey to Form 1 output.

10. TECHNOLOGY STACK SUMMARY

Component	Technology	Version / Notes
AI Model	YOLO11 Instance Segmentation	via Ultralytics + Roboflow; ONNX target
Model Training	Roboflow	Cloud training platform; 83.1% mAP@50
Backend language	Python 3.12 (laptop) / 3.11 (Pi)	Flask 3.x, Flask-CORS, python-dotenv
Camera library	OpenCV (cv2)	VideoCapture, imencode, drawing, contourArea
HTTP framework	Flask	MJPEG streaming, REST API, CORS
Pi capture script	Python requests + OpenCV	pi_capture.py — 1 fps, multipart POST
Frontend	HTML5 / Vanilla JS / CSS3	No framework; single file; no build step
Mapping	Leaflet.js 1.9.4	CARTO tiles; polylines + markers + popups
GPS (current)	Simulated (advance_gps)	Random walk ~10m steps; NEO-6M target
GPS (target)	NEO-6M + gpsd + pynmea2	UART NMEA sentences parsed on Pi
Storage (current)	Python deque (in-memory)	maxlen=300 events, lost on restart
Storage (target)	SQLite + SQLAlchemy	Persistent across reboots; exportable
PDF generation	reportlab (planned)	Form 1 Thai government repair request
Edge hardware	Raspberry Pi 4 (4GB)	Ubuntu 22.04 LTS aarch64; Pi capture + (future) inference
Development IDE	VS Code + Remote-SSH	Code on Pi from laptop; integrated terminal
Connectivity	Local Wi-Fi (dev) / 4G dongle (field)	Offline ONNX mode removes cloud dependency

11. VISION, IMPACT & FUTURE DIRECTION

Long-Term Vision

ROAD AI is designed to scale from a student competition prototype to a production tool used daily by Thai road maintenance agencies. The long-term vision is a network of Pi-equipped survey vehicles — motorcycles, pickup trucks, road maintenance crew vehicles — each streaming inspection data to a central dashboard. Municipal engineers see damage accumulate on a live map in real time, prioritise repairs by severity and cost, and generate Form 1 documents with one click instead of hours of manual paperwork.

Impact Metrics (Target)

Survey speed	Manual inspection: ~2 km/hour. ROAD AI: 40+ km/hour (highway speed)
Documentation time	Manual Form 1: 2-4 hours per survey. ROAD AI: instant auto-generation
Missed damage	Human surveys miss ~30% of early-stage cracks. AI detects consistently regardless of inspector fatigue
Cost savings	Early detection of cracks (■320/m²) prevents progression to potholes (■850/m²)
Coverage	One Pi kit costs ~■3,000—■5,000. Scalable to every municipality at low cost
Data quality	GPS-tagged, timestamped, photo-evidenced records replace handwritten notes

Design Philosophy

- Offline-first: must work on rural roads with no internet. Cloud is optional, not required.
- Low operator burden: AUTO mode requires no interaction while driving. Zero training for basic use.
- Thai context: cost tables, severity thresholds, and Form 1 output designed for Thai government requirements.
- Modular inference: swapping Roboflow → ONNX → any future model requires changing one function (infer_frame).
- Graceful degradation: if camera fails, if GPS fails, if cloud fails — each component fails independently without crashing the system.
- Transparent AI: every detection shows confidence %, class name, and physical measurements so operators can judge and override.

This document was generated automatically from the ROAD AI codebase on June 2026. It is intended as a complete context brief for AI models assisting with development of this project. All technical details are current as of the document date. For the most up-to-date state, refer to `edge/roadai.py`, `index.html`, `pi_side/pi_capture.py`, and `receiver_server.py`.